Software Requirements Specification

for

PhotoBoothique

Version 1.0 approved

Prepared by Group 4

Binus University

25 February 2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents ii
Revision History ii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description 2
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 2
2.6 User Documentation 2
2.7 Assumptions and Dependencies 3
3. External Interface Requirements 3
3.1 User Interfaces 3
3.2 Hardware Interfaces 3
3.3 Software Interfaces 3
3.4 Communications Interfaces 3
4. System Features 4
4.1 System Feature 1 4
4.2 System Feature 2 (and so on) 4
5. Other Nonfunctional Requirements 4
5.1 Performance Requirements 4
5.2 Safety Requirements 5
5.3 Security Requirements 5
5.4 Software Quality Attributes 5
5.5 Business Rules 5
6. Other Requirements 5
Appendix A: Glossary 5
Appendix B: Analysis Models 5
Appendix C: To Be Determined List 6

Revision History

Name	Date	Reason For Changes	Version
Group 3 (Albert, Jovan, Keefe)	02/28/2026	Add an introduction	1.1

Jiovanny	04/21/2026	Added Chapter 2	1.2

1. Introduction

1.1 Purpose

The purpose of this SRS document is to provide a comprehensive architectural, functional, and non-functional description of PhotoBootique (Release Version 1.0). This document serves as the single source of truth for the software's capabilities, constraints, and phased deliverables throughout its 4 month development lifecycle.

This SRS specifically covers to complete the scope of the web-based photobooth system, detailing the progression from a local camera engine to a fully cloud integrated platform with artificial intelligence capabilities. The document is designed to guide the development team through four distinct increments:

- Increment 1 (Core Engine)
  Focus: Basic camera access, capturing photos, and simple local saving.
- Increment 2 (The "Booth" Experience)
  Focus: Digital frames (overlays), filters, and basic UI/UX
- Increment 3 (Social & Cloud)
  Focus: User accounts (registration, login, authentication), cloud galley, sharing (QR codes), privacy controls.
- Increment 4 (Polish & Advanced Features)
  Focus: AI background removal, stickers, and final bug fixing / documentation

1.2 Document Conventions

This SRS follows specific typographical and structural conventions to ensure that the requirements for PhotoBoothique are clearly understood by stakeholders and developers:

Typographical Conventions

- **Boldface:** Used for core system features (e.g., Camera integration, QR Code) and primary user interface elements.
- *Italics*: Used for specific technical parameters, such as filters (*b&w*, *retro*) or non-functional constraints.
- Monospace: Used for specific time-based constraints, such as the 3-2-1 countdown timer or performance limits like 2 seconds.

1.3 Intended Audience and Reading Suggestions

This document serves as the primary roadmap for the **PhotoBoothique** project and is intended for the following stakeholders:

- **Developers**: Should focus on the **Functional Requirements** to understand camera integration, filter logic, and the QR code generation process.
- **Project Managers**: Should prioritize the **Timeline** section to track deliverables across the four planned increments.
- **UI/UX Designers**: Should pay close attention to the **Usability** constraints to ensure the interface remains "kiosk-friendly" with large, intuitive controls.
- **Testers**: Should use the requirement IDs to build test cases, specifically for performance benchmarks like the 2-second processing limit.

Reading Suggestion: New team members should start with the project overview to understand the 4-month goal, then move directly to the specific requirements that align with their technical role.

1.4 Product Scope

PhotoBoothique is an interactive, web-based software system designed to replicate and modernize the traditional physical photobooth experience. Unlike conventional photobooths that require specialized hardware or native mobile applications, PhotoBoothique operates entirely within modern web browsers (Google Chrome, Apple Safari, Microsoft Edge), making it highly accessible and easily deployable on any device with a camera and internet connection.

System Boundaries & Security:
To bridge the gap between a public kiosk experience and personal device ownership, the system utilizes an automated QR code generation mechanism. Upon completing a photo session, a unique QR code is generated. Users scan this code to securely download their processed images directly to their personal smartphones.

Because the system handles personal biometric data (user photographs), privacy and data security are the highest operational priorities. The software scope includes strict backend session isolation, where images processed on the server are temporarily held in secure, isolated memory partitions that are immediately purged after the session expires or the user downloads the file, guaranteeing zero data leakage between user sessions.

Ultimately, PhotoBoothique aligns with the goal of providing a fast (sub 2 second processing time), Secure, and highly engaging digital souvenir platform suitable for events, commercial spaces, or personal entertainment.

1.5 References

[1] IEEE. (1998). *IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications, IEEE Standar Association*
http://www.math.uaa.alaska.edu/~afkjm/cs401/IEEE830.pdf

[2] Google. (2024). Selfie Segmentation Guide. Google for Developers.
https://www.google.com/search?q=https://developers.google.com/mediapipe/solutions/vision/selfie_segmenter

[3] International Organization for Standardization. (2015). *Information technology - Automatic identification and data capture techniques - QR Code bar code symbology specification* (ISO/IEC 18004:2015). https://cdn.standards.iteh.ai/samples/83389/dee29007cb62437f9767323e6af02f90/ISO-IEC-18004-2024.pdf

2. Overall Description

2.1 Product Perspective

PhotoBoothique is a web-based photo booth app built to bring the classic photobooth experience into the browser, without needing any physical hardware. The idea came from wanting to make photo booths actually accessible, whether you're running an event, setting up at a venue, or just want something fun to use. As long as you have a device with a camera and an internet connection, it works.

There are already similar apps out there like PhotoBooth.cc.io and LittleVintagePhotoBooth, but none of them really have the full package. No account system, no cloud storage you can actually organize, no AI background removal, no stickers. PhotoBoothique was built from the ground up to put all of that in one place.

The platform handles everything in one browser-based system: real-time camera capture, photo customization, cloud storage for your gallery, and uses AI for background removal. On the backend, it connects to a cloud storage service for managing galleries and a QR code service so users can get their photos easily. There are three main types of users the system is built around: Guest Users and Registered Users, as shown in Fig. 1.

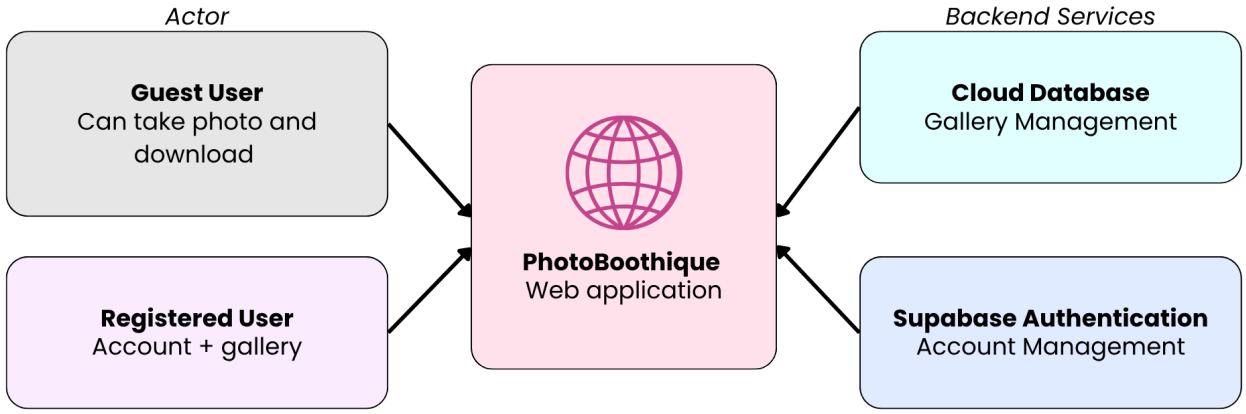


Fig. 1. Actor and Backend Services

2.2 Product Functions

The major functions of the PhotoBoothique system include:

Function	Description
Camera Access & Capture	The system accesses the device's webcam to capture photo
Photo Customization	Users can apply digital frame overlays and filters before capturing
Session Management & Privacy	Each booth session is isolated in a secure, temporary memory partition
QR Code Generation	The system generates a unique, time-limited QR code that allows users to download their processed image to a personal device securely
User Authentication & Registration	Registered users can create accounts, log in, and access a private cloud gallery of their past photo sessions
Cloud Gallery	Registered users can view, manage, and delete their stored photos
AI Background Removal	Detect and remove the photo background in under 2 seconds, allowing users to place themselves on virtual backgrounds.
Animated Stickers & Decorations	Users can place and position animated sticker overlays on their photos prior to download.

User interaction with the web application and the execution of functions can be depicted as shown in the DFD diagram in Fig. 2 and the use case diagram in Fig. 3.

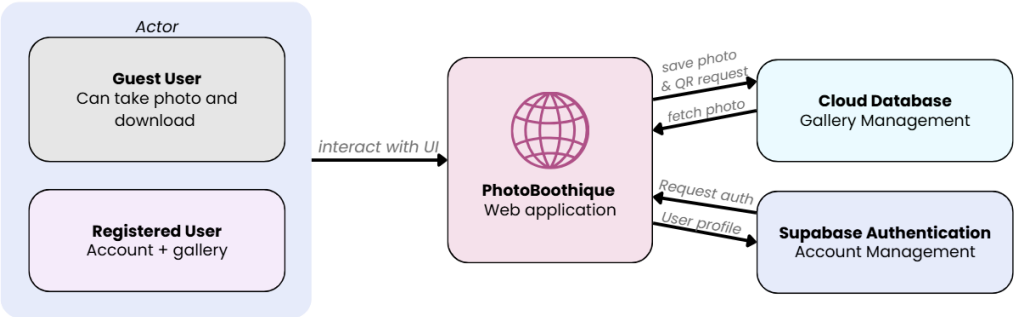


Fig. 2. Top-level Data Flow Diagram (DFD)

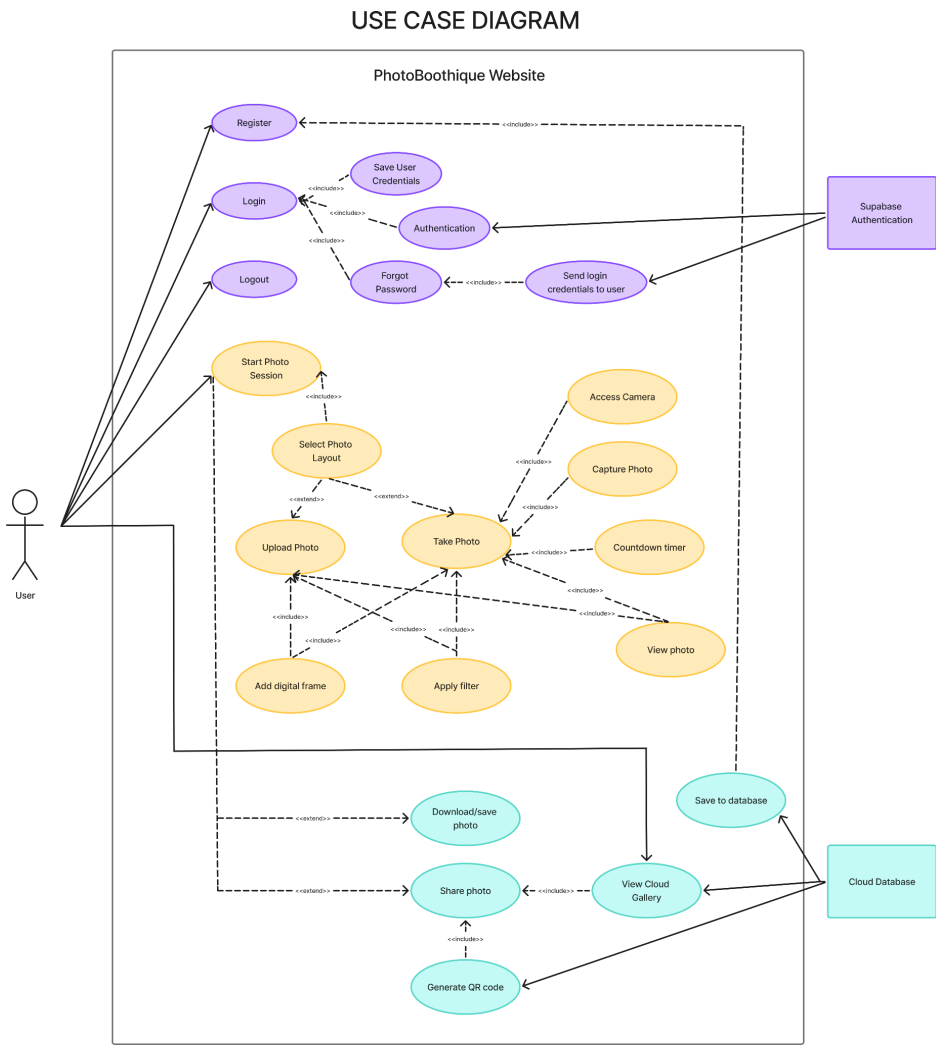


Fig. 3. Use Case Diagram

2.3 User Classes and Characteristics

The system identifies two primary user classes:

Guest User
The most frequent and casual user of the system. Guest Users do not register or log in. They access the photobooth directly, capture photos, apply filters and frames, and downloads their image. Guest Users cannot store images in a cloud gallery. They are expected to have basic ability to operate a touchscreen or browser. All their session data is ephemeral and purged upon session expiry.

Registered User
Users who create an account and authenticate with the system. Registered Users have access to all Guest User features, plus a cloud gallery, the ability to revisit and re-download past sessions. They are expected to have a valid email address and familiarity with web-based account management.

Registered Users interact with the authentication module, cloud gallery dashboard, and privacy settings.

Most important to satisfy is Guest Users because if they are satisfied the chances of them registering and being a loyal user is higher.

2.4 Operating Environment

PhotoBoothique operates as a responsive web application accessible via modern browsers on any device with a camera. The system is designed for deployment in both public and personal hardware devices (smartphones, home computers).

Component	Specification
Client Interface	Responsive web application
Supported Browsers	Google Chrome (latest 2 versions), Apple Safari (latest 2 versions), Microsoft Edge (latest 2 versions)
Camera API	W3C MediaDevices API
Backend Runtime	Node.js
Cloud Storage	Firebase Storage
Database	PostgreSQL (via Supabase)
AI Processing Library	
QR Code Generation	
Authentication	JSON Web Tokens (JWT) with OAuth 2.0 and Email
Hosting	Cloud-hosted with Vercel

2.5 Design and Implementation Constraints

Issues that limits PhotoBoothique's functions are as follows:

- **Web-Only Scope:** The current release is limited to a browser-based interface. Native iOS and Android applications are out of scope for all four increments.
- **Browser Camera Permission Dependency:** The system depends entirely on the browser's getUserMedia API. If a user denies camera permissions, the core capture feature is unavailable. The system must gracefully handle permission denial with an informative error message.
- **Client-Side AI Processing:** AI background removal is performed on the client device to avoid uploading raw biometric (photographic) data to the server. This constrains AI model complexity to those operable within typical device memory and CPU limits.

- **Quick Processing Constraint:** All photo processing operations, including filter application, background removal, and QR code generation, must complete within 2 seconds to maintain a responsive user experience.
- **Session Data Isolation:** Image data must be held in isolated, temporary server memory partitions. Persistent storage of guest session images beyond the download window is prohibited for privacy compliance.
- **Incremental Development:** Features are gated by increments. AI background removal (Increment 4) will not be available in earlier releases. All interfaces must be designed to accommodate future feature additions without full redesign.
- **Scale Constraint:** Enterprise-grade scaling (e.g., thousands of concurrent users) is not a target for this prototype.

2.6 User Documentation

The PhotoBoothique project will deliver the following documentation components:

- **Software Requirements Specification (SRS):** This document, covering all functional and non-functional requirements across all four increments.
- **User Interface Specification / Figma Prototype**: A high-fidelity design prototype covering all major screens: the booth capture view, customization panel, QR download screen, and the cloud gallery.
- **Guest User Quick Guide:** A one-page visual guide displayed on-screen during event deployments, explaining the 4-step booth process: tap to start, customize, capture, and scan to download.
- **Developer API Reference and source code:** Internal documentation and GitHub repository for the backend, authentication flow, and cloud storage integration specifications.

Our figma prototype is also available at the following link:
https://www.figma.com/proto/9qBNOVnk0ugoo04S7mzfpY/photoboothique?node-id=0-1&t=s0L68l99MjVu8oSK-1

2.7 Assumptions and Dependencies

Assumptions:

- All Guest Users are assumed to have access to a device (smartphone or tablet) capable of scanning a QR code to download their photo.
- Users are assumed to grant camera permissions willingly
- All Registered Users are assumed to have a valid email address for account creation and notification purposes.
- The device running the booth interface has a stable internet connection for cloud gallery sync, QR code delivery, and AI model loading.

Dependencies:

- W3C MediaDevices API (getUserMedia): The core capture feature depends entirely on this browser API. Functionality is unavailable in browsers or environments that do not support it.
- AI Library: AI background removal depends on the tools used. Network access is required for initial model loading; subsequent processing is client-side.
- Cloud Storage Service (Firebase): The Registered User gallery and QR code delivery system depend on an available cloud storage backend. Service outages will degrade gallery and download features.
- Third-Party Authentication: Social login (e.g., Google OAuth) depends on the availability of the respective identity provider.
- QR Code Library: The download link delivery mechanism depends on a functional QR code generation library integrated into the backend.

3. External Interface Requirements

3.1 User Interfaces

The PhotoBoothique interface is designed to be kiosk-friendly, touch-optimized, and immediately intuitive, even for first-time users who have never interacted with the application before. Because the system may be deployed on event kiosk tablets in noisy, high-distraction environments, the UI must prioritize large touch targets, minimal cognitive load, and a visually engaging aesthetic.

GUI Standards and Product Style Guide:

- Design Language: The interface shall follow a modern, playful design language inspired by Material Design 3 principles, adapted to a fun and celebratory photobooth aesthetic. Cards, rounded corners, and bold brand colors are the primary visual motif.
- Color Palette: The primary brand color deep pink (#FFACD4) shall be used for primary call-to-action buttons (Capture, Download, Apply).
- Typography: A bold, rounded Poppins and Disney font shall be used globally. Large text sizes (minimum 18pt for body, 24pt for action labels) are required for legibility in bright event lighting conditions.

Screen Layout Constraints:

- Responsive Design: All interfaces must be responsive from a minimum viewport width of 375px (mobile) up to 1920px (desktop kiosk), using CSS flexbox or grid layouts.
- Touch Target Sizing: All interactive elements (buttons, filter thumbnails, frame selection tiles) must have a minimum touch target size of 48x48 CSS pixels, compliant with WCAG 2.1 AA guidelines.
- Camera Preview Dominance: On the booth capture screen, the live camera preview must occupy no less than 60% of the visible viewport area to provide a clear, engaging preview for the user.
- Navigation: A minimal, icon-driven bottom navigation shall be used for authenticated users (Gallery, Booth, Account). Guest Users see a single-flow linear interface with no persistent navigation.

Below is an example of the UI made in Figma:

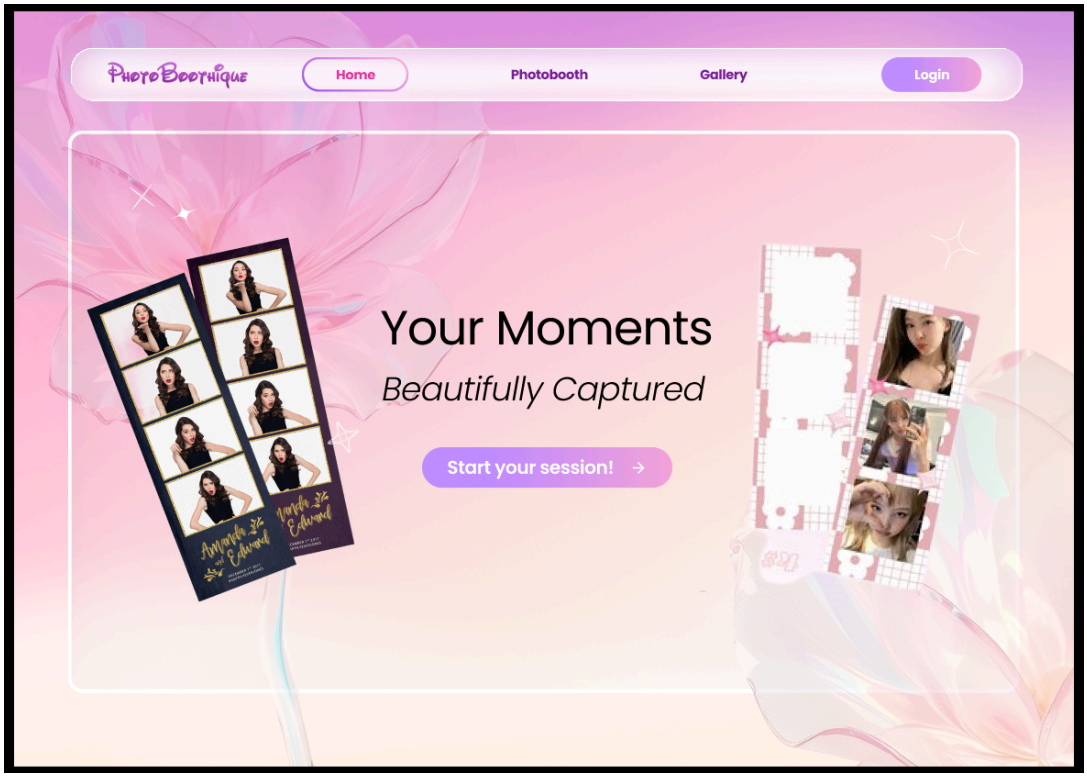


Image 1. Landing Page

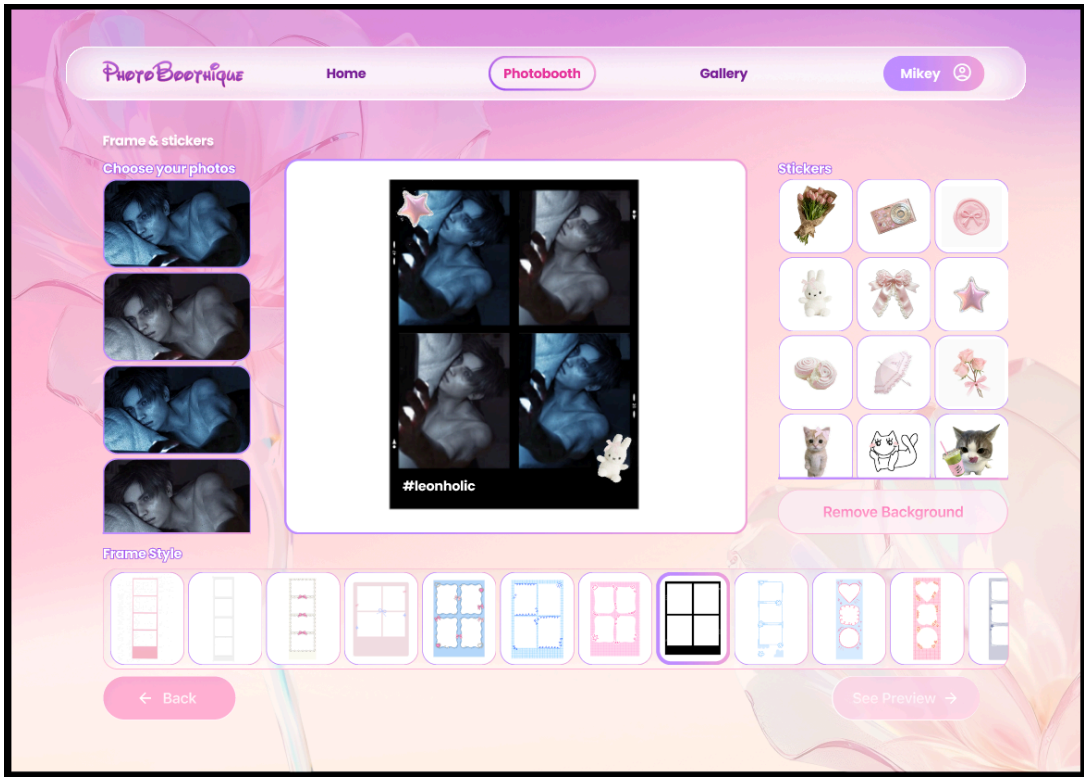


Image 2. Edit Frame and Stickers

Error Message Display Standards:

- All system errors, validation failures, and success confirmations shall be displayed as Toast Notifications at the top or bottom of the screen, auto-dismissing after 3 to 4 seconds.
- Success: Green background with checkmark icon (e.g., "Photo saved to your gallery!").
- Error / Warning: Red or amber background with alert icon (e.g., "Camera access denied. Please allow camera permissions in your browser settings.").
- Critical blocking errors (e.g., no camera detected) shall be displayed as a full-screen modal that prevents the user from proceeding until the issue is resolved.

Software Components Requiring a UI:

- Booth Capture Screen: The primary screen showing the live camera feed with overlaid frames, a filter strip carousel below the preview, a Capture button, and a countdown timer display.
- Customization Panel: A sliding drawer panel where users select frame overlays and filters before capture. Includes thumbnail previews for each option.
- QR Code Download Screen: Displays the generated QR code prominently, a countdown timer showing how long the code remains valid, and an option to take another photo.
- Authentication Screens: Login, Registration (name, email, password), and Forgot Password screens.
- Cloud Gallery Dashboard: A masonry or grid-based gallery view of the Registered User's saved photos with options to view, download, toggle privacy, and delete.

3.2 Hardware Interfaces

PhotoBoothique is a web application and does not require proprietary external hardware. It operates on standard consumer-grade devices through their web browsers using the W3C MediaDevices API.

Interface	Specification
Supported Form Factors	Smartphones (portrait orientation), tablets (portrait and landscape), laptops, desktop computers, kiosk touchscreen displays
Camera Hardware	Front-facing or rear-facing camera accessible via the browser's MediaDevices API (getUserMedia). Minimum resolution: 720p (1280x720) for acceptable photo quality.
Minimum Device Requirements	Any device capable of running a modern Chromium-based or Safari browser with an active internet connection and camera hardware.
Input Devices	Touchscreen (primary for kiosk/mobile deployment), mouse and keyboard (desktop)
Network Interface	Wi-Fi or cellular data (minimum 4G / 10 Mbps) for cloud gallery synchronization and AI model loading. Core capture features operate with degraded-mode offline support.

Output Devices	Device display (primary output), QR-code-equipped smartphone screen (for the end-user photo delivery)

3.3 Software Interfaces

PhotoBoothique integrates with the following software components and external services:

Layer / Component	Technology / Interface	Purpose
Frontend Framework	React.js or Vue.js	Manages UI state, camera preview rendering, real-time filter application, and client-side routing across all screens
Backend / API Layer	Node.js with Express.js	Handles business logic, authentication, QR code generation, session management, and cloud storage communication
Database	PostgreSQL via Supabase or Firebase Firestore	Stores user accounts, session metadata, gallery photo references, and event operator configurations
Camera Access	W3C MediaDevices API (getUserMedia)	Browser-native API for accessing the device camera stream — no external SDK required
AI Processing	Google MediaPipe Selfie Segmenter (WASM)	Client-side real-time background segmentation and removal; model loaded once per session
Cloud Storage	Firebase Storage or AWS S3	Stores processed photo files for Registered User galleries; issues time-limited signed URLs for QR code downloads
QR Code Generation	qrcode.js or qrcode-generator (Node.js)	Generates unique, time-limited QR codes encoding the signed download URL for a completed photo session
Authentication	JWT + OAuth 2.0 (Google)	Manages user sessions; JWT tokens issued on login with 24-hour expiry; Google OAuth for social login
Canvas Processing	HTML5 Canvas API	Client-side compositing of the captured frame with selected overlay and filter before upload or local download
Data Exchange Format	JSON	Used for all client-server API communication

3.4 Communications Interfaces

- Protocol: All client-server communication must occur over HTTPS (TLS 1.2 or higher) to protect photo data and user credentials in transit. HTTP requests must be forcefully redirected to HTTPS.
- Architecture: The system uses a RESTful API architecture. Standard HTTP methods are applied semantically: GET for retrieving gallery photos and session metadata, POST for submitting captured photos and creating user accounts, PUT/PATCH for updating photo privacy settings and account details, and DELETE for removing gallery photos and session records.
- Camera Stream: The live camera preview is handled entirely client-side via the MediaStream API and rendered on an HTML5 <canvas> or <video> element. No raw camera stream data is transmitted to the server.
- Image Upload: Processed image data (post-compositing) is transmitted to the backend as a Base64-encoded payload or as multipart/form-data, and then forwarded to cloud storage. Maximum upload payload per photo session is capped at 5MB.
- QR Code Delivery: The QR code encodes a time-limited signed URL (expiry: 15 minutes) pointing directly to the processed image in cloud storage. This URL is generated server-side and transmitted to the client as a JSON response.
- Real-Time Constraints: The camera preview must operate at a minimum of 24 frames per second to appear smooth. AI background removal processing must complete within 2 seconds of capture confirmation.
- Auto-Sync for Registered Users: For registered users with calendar-like periodic refresh, the gallery dashboard shall refresh the photo list on page load and upon user-triggered pull-to-refresh actions.

4. System Features

<This template illustrates organizing the functional requirements for the product by system features, the major services provided by the product. You may prefer to organize this section by use case, mode of operation, user class, object class, functional hierarchy, or combinations of these, whatever makes the most logical sense for your product.>

4.1 System Feature 1

<Don't really say "System Feature 1." State the feature name in just a few words.>

4.1.1 Description and Priority

<Provide a short description of the feature and indicate whether it is of High, Medium, or Low priority. You could also include specific priority component ratings, such as benefit, penalty, cost, and risk (each rated on a relative scale from a low of 1 to a high of 9).>

4.1.2 Stimulus/Response Sequences

<List the sequences of user actions and system responses that stimulate the behavior defined for this feature. These will correspond to the dialog elements associated with use cases.>

4.1.3 Functional Requirements

<Itemize the detailed functional requirements associated with this feature. These are the software capabilities that must be present in order for the user to carry out the services provided by the feature, or to execute the use case. Include how the product should respond to anticipated error conditions or invalid inputs. Requirements should be concise, complete, unambiguous, verifiable, and necessary. Use "TBD" as a placeholder to indicate when necessary information is not yet available.>

<Each requirement should be uniquely identified with a sequence number or a meaningful tag of some kind.>

REQ-1:
REQ-2:

4.2 System Feature 2 (and so on)

5. Other Nonfunctional Requirements

5.1 Performance Requirements

<If there are performance requirements for the product under various circumstances, state them here and explain their rationale, to help the developers understand the intent and make suitable design choices. Specify the timing relationships for real time systems. Make such requirements as specific as possible. You may need to state performance requirements for individual functional requirements or features.>

5.2 Safety Requirements

<Specify those requirements that are concerned with possible loss, damage, or harm that could result from the use of the product. Define any safeguards or actions that must be taken, as well as actions that must be prevented. Refer to any external policies or regulations that state safety issues that affect the product's design or use. Define any safety certifications that must be satisfied.>

5.3 Security Requirements

<Specify any requirements regarding security or privacy issues surrounding use of the product or protection of the data used or created by the product. Define any user identity authentication

requirements. Refer to any external policies or regulations containing security issues that affect the product. Define any security or privacy certifications that must be satisfied.>

5.4 Software Quality Attributes

<Specify any additional quality characteristics for the product that will be important to either the customers or the developers. Some to consider are: adaptability, availability, correctness, flexibility, interoperability, maintainability, portability, reliability, reusability, robustness, testability, and usability. Write these to be specific, quantitative, and verifiable when possible. At the least, clarify the relative preferences for various attributes, such as ease of use over ease of learning.>

5.5 Business Rules

<List any operating principles about the product, such as which individuals or roles can perform which functions under specific circumstances. These are not functional requirements in themselves, but they may imply certain functional requirements to enforce the rules.>

6. Other Requirements

<Define any other requirements not covered elsewhere in the SRS. This might include database requirements, internationalization requirements, legal requirements, reuse objectives for the project, and so on. Add any new sections that are pertinent to the project.>

Appendix A: Glossary

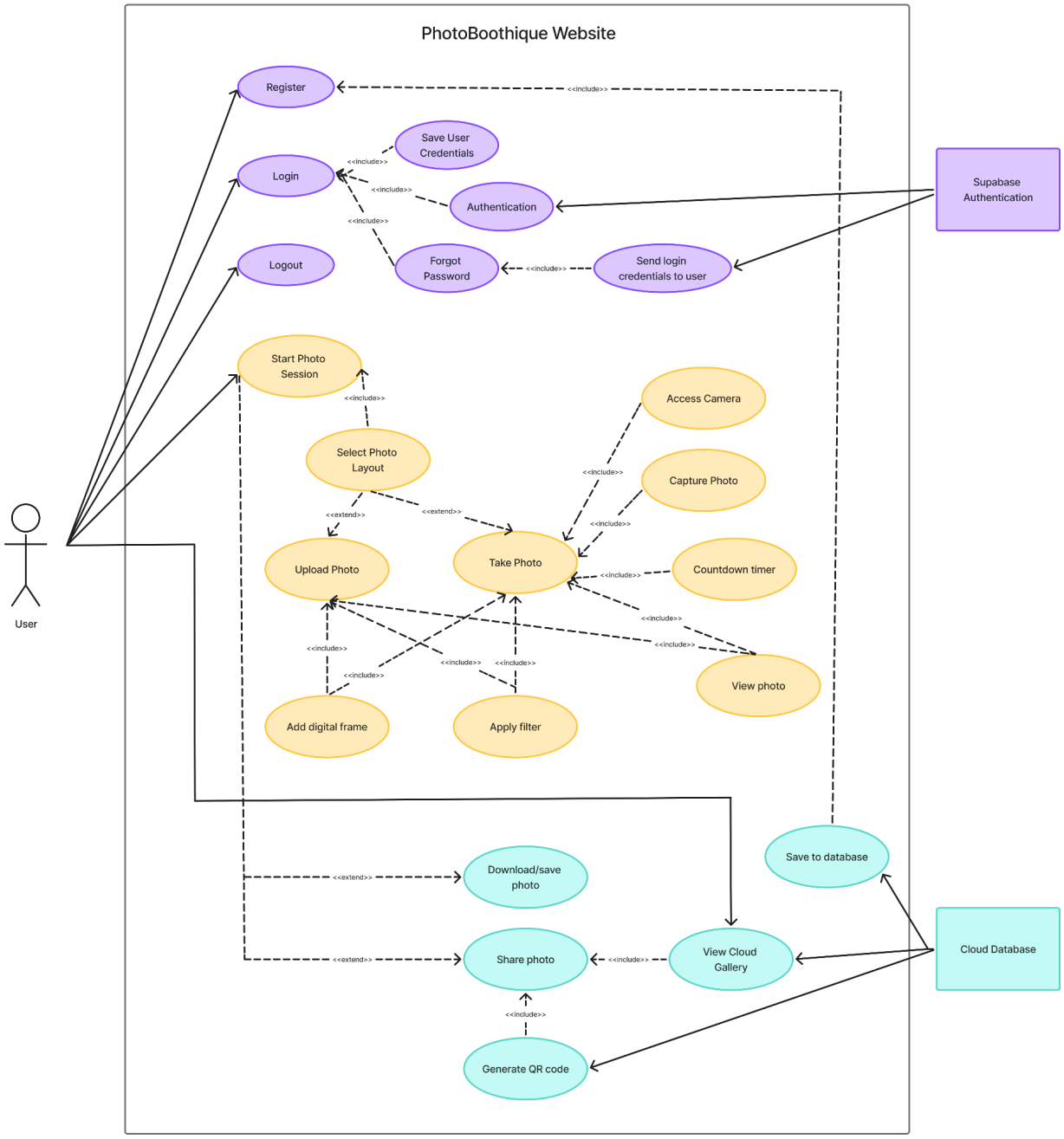
<Define all the terms necessary to properly interpret the SRS, including acronyms and abbreviations. You may wish to build a separate glossary that spans multiple projects or the entire organization, and just include terms specific to a single project in each SRS.>

Appendix B: Analysis Models

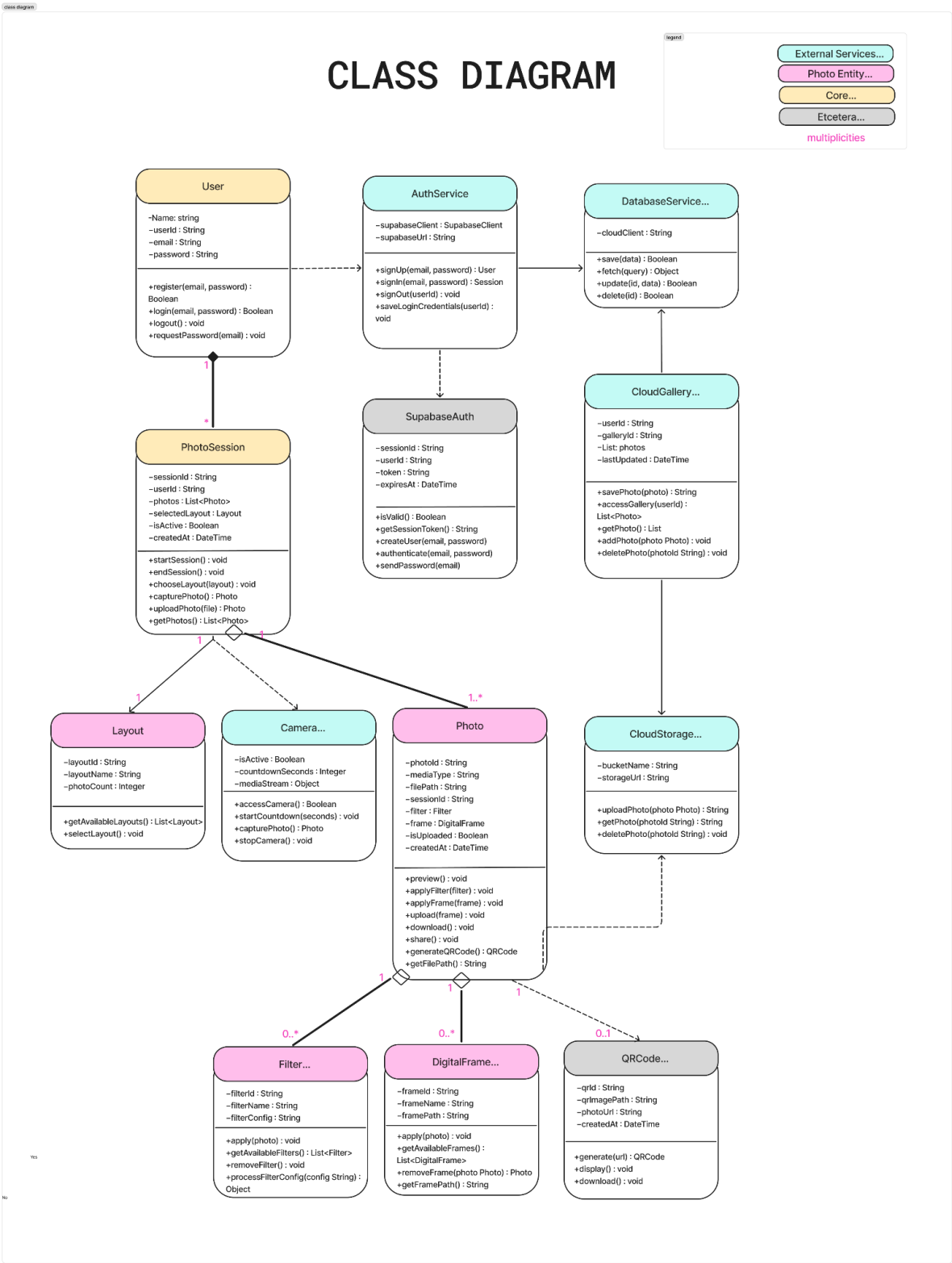
All diagrams can be accessed through this link:
https://www.figma.com/board/puMNxfNmxnywbomPM37gVP/Software-Engineering-PhotoBoothique?node-id=0-1&t=8OTHURuybNE6NNFw-1

1. Use case

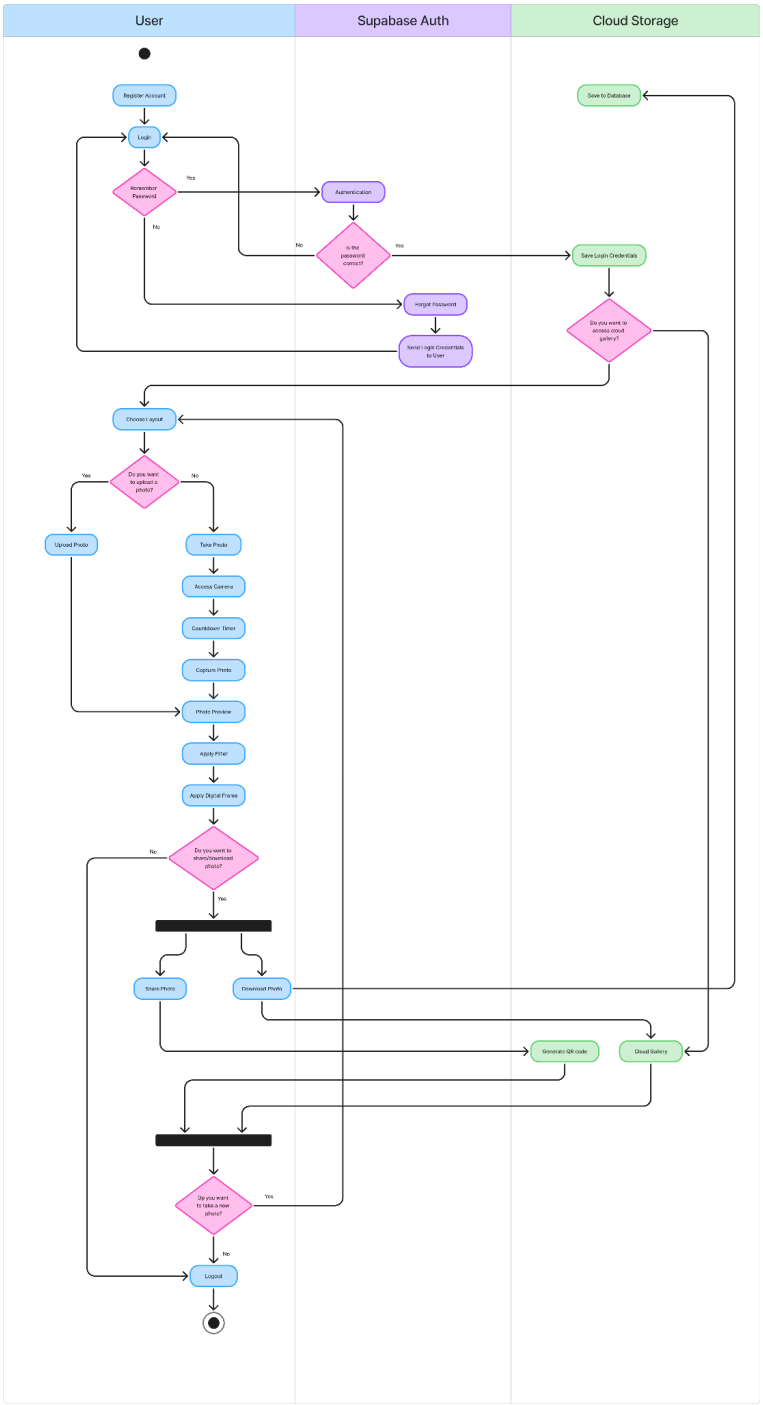
USE CASE DIAGRAM



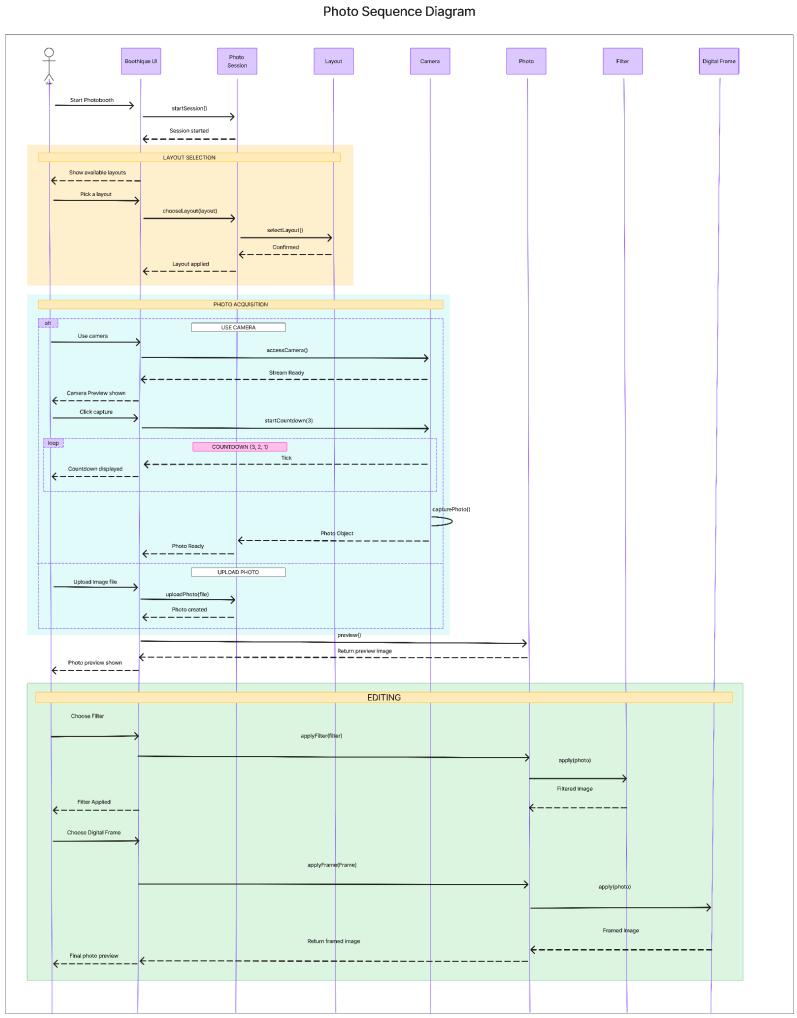
2. Class Diagram



3. Activity Diagram

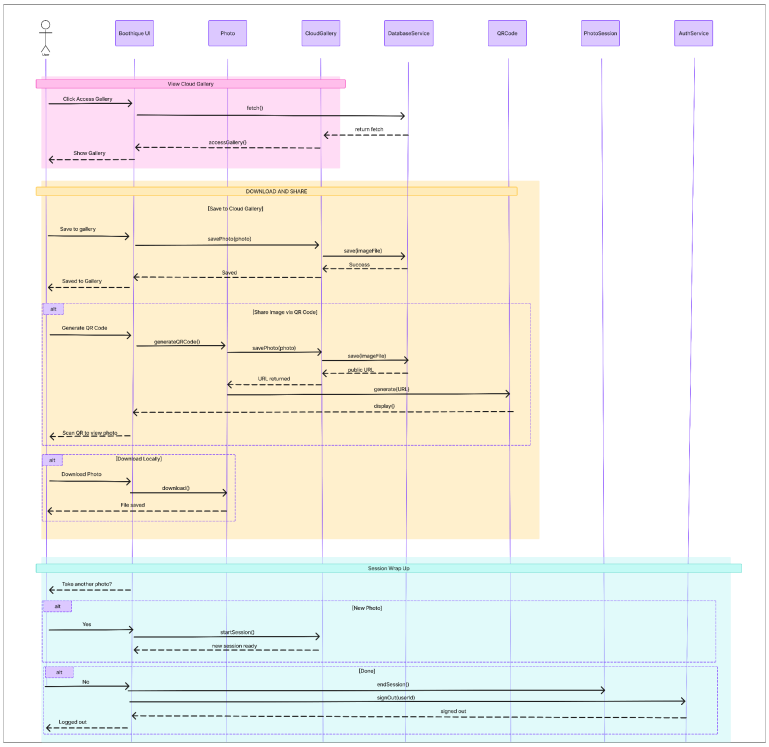


4. Sequence Diagram
    a. Photo Sequence Diagram



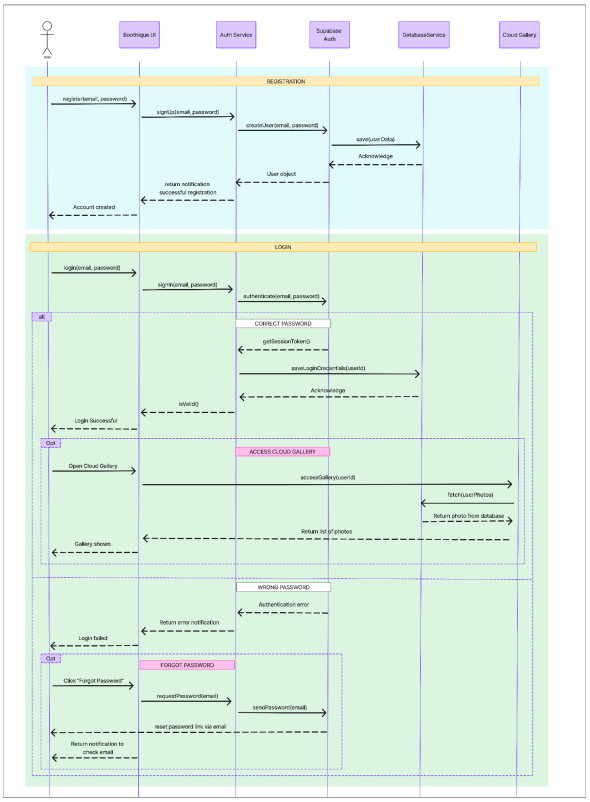
b. View, Download, Share Sequence Diagram

View, Download, Share Sequence Diagram



c. Authentication Sequence Diagram

Authentication Sequence Diagram



Appendix C: To Be Determined List

<Collect a numbered list of the TBD (to be determined) references that remain in the SRS so they can be tracked to closure.>